

RÉPUBLIQUE TUNISIENNE
Ministère de l'Enseignement Supérieur et de la
Recherche Scientifique
Établissement d'enseignement supérieur privé
Ecole Supérieure Privée de Technologie et Ingénierie

RAPPORT DE PROJET DE DÉVELOPPEMENT

Spécialité : Génie Logiciel et Systèmes d'Information

Réalisé par

**Wassim Masmoudi
Souheil Moussa
Maila Kdous
Yasmine Riahi**

PLANORA

Supervisé par :

Mme. Sonia Gharsalli

Mr. Mohamed Alouani

Réalisé au sein de TEKUP



Année Universitaire : 2025 / 2026

Table des matières

1	Contexte général	3
1.1	Présentation du cadre du projet	3
1.1.1	Etude de l'existant	3
1.1.2	Tableau comparatif des solutions existantes	4
1.1.3	Critique de l'existant	5
1.1.4	Solution proposée	6
1.2	Choix Méthodologique	6
1.2.1	Méthodologie du modèle en cascade	7
1.2.2	Méthodologie agile	8
1.3	Langage de Modélisation	10
1.3.1	Présentation de UML	10
1.3.2	Diagrammes UML	11
2	Analyse et Spécification des besoins	12
2.1	Spécification des besoins	12
2.1.1	Identification des acteurs	12
2.1.2	Besoins fonctionnels	13
2.1.3	Besoins non fonctionnels	15
2.2	Diagramme de cas d'utilisation global	16
2.3	Raffinement des cas d'utilisation	16
2.3.1	Gestion des utilisateurs et authentification	16
2.3.2	Gestion des espaces de travail	18
2.3.3	Gestion des projets	19
2.3.4	Gestion du backlog, des tâches et des sprints	19
2.3.5	Gestion de la collaboration et des réunions	20
2.3.6	Suivi de la santé de l'équipe	21

3	Conception	23
3.1	Outil de conception	23
3.1.1	StarUML	23
3.2	Conception détaillée	24
3.2.1	Diagramme de classes général	24
3.2.2	Gestion de l'authentification et des utilisateurs	26
3.2.3	Gestion des espaces de travail et des projets	26
3.2.4	Gestion du backlog, des tâches et des sprints	27
3.2.5	Gestion de la collaboration et des réunions	27
3.2.6	Suivi de la santé de l'équipe par intelligence artificielle	28
4	Réalisation	29
4.1	Environnement de développement	29
4.1.1	SQL Server Management Studio	29
4.1.2	Visual Studio	30
4.1.3	Visual Studio Code	30
4.1.4	Postman	30
4.2	Environnement logiciel	31
4.2.1	Microsoft SQL Server	31
4.2.2	Dotnet	31
4.2.3	LaTeX	32
4.3	Pilotage du cycle de vie de développement avec Scrum	32
4.3.1	Équipe Scrum et rôles	32
4.4	Architecture technique	33
4.4.1	Architecture générale	33
4.4.2	Architecture de l'environnement	34
4.4.3	Tests et validation	35
4.4.4	Backlog du produit	36
4.5	Sprints de réalisation	39
4.5.1	Sprint 1 - Authentification et gestion des utilisateurs	40
4.5.2	Sprint 2 - Gestion des espaces de travail et des projets	41
4.5.3	Sprint 3 - Gestion agile : backlog, tâches et sprints	42
4.5.4	Sprint 4 - Collaboration, réunions et tableau de bord	44

4.5.5	Sprint 5 - Suivi de la santé de l'équipe par IA	45
	Bibliographie	47

Table des figures

1.1	Méthodologie Hybride adoptée[1]	7
1.2	Cycle de vie de développement d'un logiciel agile	8
1.3	Processus de la méthode Scrum	10
2.1	Diagramme de cas d'utilisation globale	16
2.2	Diagramme de cas d'utilisation « Gestion des utilisateurs et authentification »	16
2.3	Diagramme de cas d'utilisation « Gestion des espaces de travail »	18
2.4	Diagramme de cas d'utilisation « Gestion des projets »	19
2.5	Diagramme de cas d'utilisation « Gestion du backlog, des tâches et des sprints »	19
2.6	Diagramme de cas d'utilisation « Gestion de la collaboration et des réunions »	20
2.7	Diagramme de cas d'utilisation « Suivi de la santé de l'équipe »	21
3.1	Logo de StarUML	23
3.2	Diagramme de classes général de l'application Planora	26
3.3	Diagramme de séquence de la gestion de l'authentification et des utilisateurs	26
3.4	Diagramme de séquence de la gestion des espaces de travail et des projets	27
3.5	Diagramme de séquence de la gestion du backlog, des tâches et des sprints	27
3.6	Diagramme de séquence de la gestion de la collaboration et des réunions	28
3.7	Diagramme de séquence du suivi de la santé de l'équipe par intelligence artificielle	28
4.1	Logo de SQL Server Management Studio	30
4.2	Logo de Visual Studio	30
4.3	Logo de Visual Studio Code	30
4.4	Logo de Postman	31
4.5	Logo de Microsoft SQL Server	31
4.6	Logo de Dotnet	31
4.7	Logo de LaTeX	32

4.8	Équipe Scrum et rôles	33
4.9	Architecture générale	33
4.10	Processus de test des points d'accès sécurisés par Keycloak	36
4.11	Sprint 1 - Interface de connexion	41
4.12	Sprint 1 - Interface d'inscription	41
4.13	Sprint 1 - Interface de gestion des utilisateurs	41
4.14	Sprint 2 - Interface des espaces de travail	42
4.15	Sprint 2 - Interface de gestion des projets	42
4.16	Sprint 3 - Interface de gestion du backlog	43
4.17	Sprint 3 - Tableau de sprint	44
4.18	Sprint 4 - Interface de messagerie	45
4.19	Sprint 4 - Calendrier des réunions	45
4.20	Sprint 5 - Interface de tableau de bord	45
4.21	Sprint 5 - Interface de suivi de la santé de l'équipe	46
4.22	Sprint 5 - Test du service IA d'analyse des sentiments	46

Liste des tableaux

1.1	Comparaison entre les solutions existantes et Planora	4
1.2	Types des diagrammes UML	11
2.1	Identification des Acteurs et leurs Fonctions dans le Système Applicatif . .	13
2.2	Description textuelle du cas d'utilisation « S'authentifier »	17
2.3	Description textuelle du cas d'utilisation « Gérer les utilisateurs »	18
2.4	Description textuelle du cas d'utilisation « Gérer un espace de travail » . .	18
2.5	Description textuelle du cas d'utilisation « Gérer un projet »	19
2.6	Description textuelle du cas d'utilisation « Gérer le backlog et les sprints »	20
2.7	Description textuelle du cas d'utilisation « Collaborer au sein d'un projet »	21
2.8	Description textuelle du cas d'utilisation « Analyser la santé de l'équipe » .	22
4.1	Légende des priorités du backlog	37
4.2	Backlog du produit et priorisation des tâches	37
4.3	Backlog du Sprint 1 : Authentification et gestion des utilisateurs	40
4.4	Backlog du Sprint 2 : Gestion des espaces de travail et des projets	41
4.5	Backlog du Sprint 3 : Gestion agile	43
4.6	Backlog du Sprint 4 : Collaboration et suivi	44
4.7	Backlog du Sprint 5 : Suivi de la santé de l'équipe par IA	45

Liste des abréviations

SSMS	SQL Server Management Studio
VS	Visual Studio
VSCode	Visual Studio Code
API	Application Programming Interface
JWT	JSON Web Token
SQL	Structured Query Language
JSON	JavaScript Object Notation
REST	Representational State Transfer
ASPNET	ASP.NET
DOTNET	.NET
DTO	Data Transfer Object
EF	Entity Framework
SignalR	SignalR
IA	Intelligence Artificielle
UML	Unified Modeling Language

Introduction générale

De nos jours, la gestion de projet et la collaboration au sein des équipes constituent des enjeux essentiels pour les organisations. Face à la complexité croissante des projets, à la multiplication des tâches et à la nécessité d'assurer une bonne coordination entre les membres, l'utilisation d'outils numériques adaptés devient indispensable. Les équipes de développement logiciel, en particulier, ont besoin de solutions permettant de planifier les travaux, suivre l'avancement, gérer les priorités et favoriser une communication efficace.

Planora se présente comme une application web de gestion de projet conçue pour répondre aux besoins des équipes travaillant selon une approche collaborative et agile. Elle permet de centraliser les projets, les espaces de travail, les tâches, les backlogs, les sprints, les réunions et les échanges entre les membres. L'application intègre également des fonctionnalités intelligentes liées à l'analyse de la santé de l'équipe, afin d'aider les responsables à mieux comprendre l'état général du groupe et à détecter les signes de stress ou de frustration.

C'est dans ce cadre que s'inscrit ce projet de fin d'études, visant à concevoir et développer une plateforme complète de gestion de projet nommée Planora. Ce projet englobe la mise en place d'une architecture web basée sur un backend .NET (DOTNET), un frontend Angular, ainsi qu'un module d'intelligence artificielle dédié à l'analyse des sentiments à partir des messages échangés entre les membres de l'équipe. L'objectif principal est de proposer une solution centralisée, ergonomique et évolutive permettant d'améliorer l'organisation, le suivi et la collaboration au sein des projets.

Le présent rapport est articulé autour des chapitres suivants :

- Dans le premier chapitre, intitulé « Contexte général », nous présentons le cadre général du projet Planora, son domaine d'application, l'étude de l'existant ainsi que la méthodologie de gestion de projet retenue.
- Dans le deuxième chapitre, nommé « Analyse et spécification des besoins », nous identifions les acteurs du système et définissons les besoins fonctionnels et non fonctionnels. Nous y détaillons les principales fonctionnalités de l'application à travers des diagrammes de cas d'utilisation.

-
- Dans le troisième chapitre, « Conception », nous présentons l'architecture générale de la solution, la modélisation des données, les diagrammes de classes et les principaux scénarios d'interaction entre les différents composants du système.
 - Dans le quatrième et dernier chapitre, intitulé « Réalisation », nous décrivons l'environnement de développement, les outils et technologies employés, ainsi que les interfaces les plus représentatives de l'application réalisée.

Ce rapport se clôturera par une conclusion synthétisant les acquis du projet et mettant en lumière les perspectives d'amélioration envisageables pour enrichir davantage les fonctionnalités de Planora.

1

Contexte général

Introduction

Ce chapitre introductif détaillera l'étude de l'existant, la problématique associée, ainsi qu'une évaluation approfondie de la solution proposée et des objectifs globaux du projet. Enfin, la dernière partie définira la méthodologie de gestion de projet adoptée.

1.1 Présentation du cadre du projet

Dans cette section, nous fournirons une analyse critique de l'existant, ainsi qu'une présentation détaillée de la solution proposée.

1.1.1 Etude de l'existant

Avant le développement de l'application Planora, une étude de l'existant a été réalisée afin d'analyser les solutions déjà disponibles sur le marché dans le domaine de la gestion de projets et du suivi collaboratif. Plusieurs plateformes et applications web proposent des fonctionnalités similaires, telles que la création de projets, la gestion des tâches, l'organisation des sprints, le suivi du backlog ou encore la collaboration entre les membres d'une équipe.

Parmi les solutions existantes, on retrouve des outils de gestion de projet comme Jira, Trello, Asana ou Monday.com, ainsi que des plateformes orientées vers les méthodes agiles et la communication d'équipe. Certaines de ces solutions intègrent également des tableaux de bord, des calendriers, des espaces de discussion, des systèmes de notification et des fonctionnalités d'analyse pour accompagner les équipes dans le suivi de leur avancement. Cette étude a permis d'identifier les fonctionnalités communes, les bonnes pratiques ainsi que les limites techniques et fonctionnelles des solutions existantes, notamment en matière de centralisation des données, de suivi de la santé de l'équipe et d'intégration de fonctionnalités intelligentes d'aide à la décision.

1.1.2 Tableau comparatif des solutions existantes

Le tableau 1.1 présente une comparaison entre quelques solutions existantes de gestion de projet et la solution proposée Planora.

Dans ce tableau, la mention « Partiel » indique que la fonctionnalité existe de manière limitée ou nécessite une intégration avec un outil externe.

Tab. 1.1 : Comparaison entre les solutions existantes et Planora

Solution	Tâches, backlog et sprints	Chat	Réunions et calendrier	Analyse intelligente
Jira	Oui	Partiel : via intégration avec Slack ou Teams	Partiel : via intégration avec Google Calendar ou Outlook	Non
Trello	Partielle : gestion simple des cartes, sans sprints avancés natifs	Partiel : via commentaires ou intégrations externes	Limitée : calendrier disponible selon les vues et extensions	Non

Solution	Tâches, backlog et sprints	Chat	Réunions et calendrier	Analyse intelligente
Asana	Partielle : gestion des tâches avancée mais backlog et sprints non spécialisés	Partiel : via commentaires ou intégrations externes	Oui	Non
Monday.com	Partielle : gestion personnalisable mais agile avancé selon configuration	Oui	Oui	Non
Planora	Oui	Oui	Oui	Oui : analyse des messages pour détecter le stress, la frustration et l'ambiance de l'équipe

1.1.3 Critique de l'existant

Malgré la diversité des solutions existantes, plusieurs insuffisances ont été constatées. La majorité des plateformes de gestion de projet proposent des fonctionnalités intéressantes, telles que la gestion des tâches, la planification des sprints ou le suivi de l'avancement, mais elles restent parfois dispersées entre plusieurs outils, ce qui complique le travail des équipes et limite la centralisation des informations.

De plus, certaines applications manquent de simplicité dans leur utilisation, notamment pour les équipes qui souhaitent adopter rapidement une méthode agile sans configuration complexe. L'absence d'un suivi précis de la santé de l'équipe constitue également une limite importante, car la plupart des solutions se concentrent principalement sur l'avancement technique du projet sans prendre en compte l'état général des membres, le stress ou la frustration exprimés pendant la collaboration.

Par ailleurs, les fonctionnalités de communication, de gestion des réunions et d'aide intelligente à la décision ne sont pas toujours intégrées dans une même plateforme, ce qui oblige les utilisateurs à utiliser plusieurs outils en parallèle. Ces limites soulignent la nécessité de concevoir une solution plus complète, centralisée et centrée sur les besoins réels des équipes de projet.

1.1.4 Solution proposée

Afin de remédier aux limites identifiées, le projet Planora propose une solution web complète regroupant plusieurs services de gestion de projet au sein d'une seule plateforme. L'application permet aux utilisateurs de créer un compte sécurisé, de gérer des espaces de travail, de créer des projets, d'inviter des membres et d'organiser les tâches selon une approche collaborative et agile. Elle offre également la possibilité de gérer le backlog, les sprints, les sous-tâches, les réunions ainsi que le suivi de l'avancement à travers des tableaux de bord.

L'intégration d'un module basé sur l'intelligence artificielle constitue un élément différenciateur majeur, permettant d'analyser les messages échangés entre les membres afin d'évaluer la santé de l'équipe et de détecter les signes de stress ou de frustration. De plus, la plateforme intègre un espace de messagerie favorisant la communication et la coordination entre les membres du projet.

Cette solution vise à améliorer l'organisation, la collaboration et le suivi des projets tout en assurant une expérience utilisateur cohérente, centralisée et adaptée aux besoins réels des équipes.

1.2 Choix Méthodologique

Il est primordial d'opter pour la bonne méthodologie avant de débiter un projet informatique. En effet, aucune méthode universelle ne peut garantir à elle seule l'exécution adéquate de l'ensemble des tâches d'un projet, tant les contextes et les objectifs varient d'un projet à l'autre.

La méthode sélectionnée doit donc être adaptée aux spécificités et aux contraintes de chaque projet. Ainsi, il est essentiel de retenir une approche qui réponde aux besoins réels des parties prenantes, afin de garantir un résultat optimal et conforme aux attentes.

Dans le présent contexte, nous avons opté pour une approche de gestion de projet hybride, connue sous le nom de modèle **Water-Scrum-Fall**, qui combine les avantages des deux méthodologies les plus répandues :

- La planification rigoureuse et la documentation exhaustive du **modèle en cascade** lors des phases d'audit et de conception, en s'appuyant sur Unified Modeling Language (UML) comme langage de modélisation.
- Le développement itératif et incrémental de la méthodologie **Scrum** lors des phases de mise en place et de développement, organisé en sprints permettant des livraisons régulières et une adaptation continue aux retours obtenus.

La figure 1.1 ci-dessous illustre la méthodologie hybride adoptée :

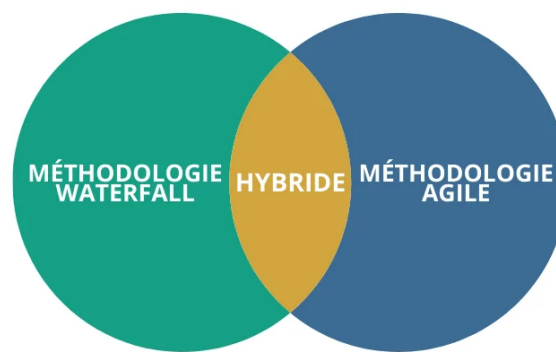


Fig. 1.1 : Méthodologie Hybride adoptée[1]

En tirant parti des forces de chaque méthodologie, cette approche hybride vise à maximiser l'efficacité, à atténuer les risques et à assurer une livraison progressive et contrôlée des différents composants du projet.

1.2.1 Méthodologie du modèle en cascade

Le modèle en cascade, caractérisé par son approche séquentielle, est souvent associé lors de la phase de conception dans nombreux projets. Cette approche garantit une analyse approfondie des exigences et des objectifs du projet, suivie par la conception d'une solution complète et détaillée répondant à ces exigences.

Cette conception est ensuite examinée et validée avant de passer à la phase de développement proprement dite.

La linéarité du processus en cascade permet une gestion efficace des ressources et une planification minutieuse des étapes, offrant une structure et une prévisibilité précieuses dans la phase d'analyse des besoins et de conception[2]

1.2.2 Méthodologie agile

La méthodologie Agile adoptée durant le cycle de vie de développement se distingue des approches traditionnelles telles que le cycle en V ou le modèle en cascade, qui nécessitent une planification complète du projet avant le début du développement, par sa flexibilité et son adaptabilité en se concentrant sur des objectifs à court terme.

Plaçant le client au cœur du processus, cette approche favorise une collaboration étroite et des retours réguliers, ce qui permet à l'équipe de développement d'intégrer rapidement les modifications nécessaires pour répondre aux évolutions du projet[3]. La figure 1.2 ci-dessous illustre le cycle de vie de développement d'un logiciel agile :

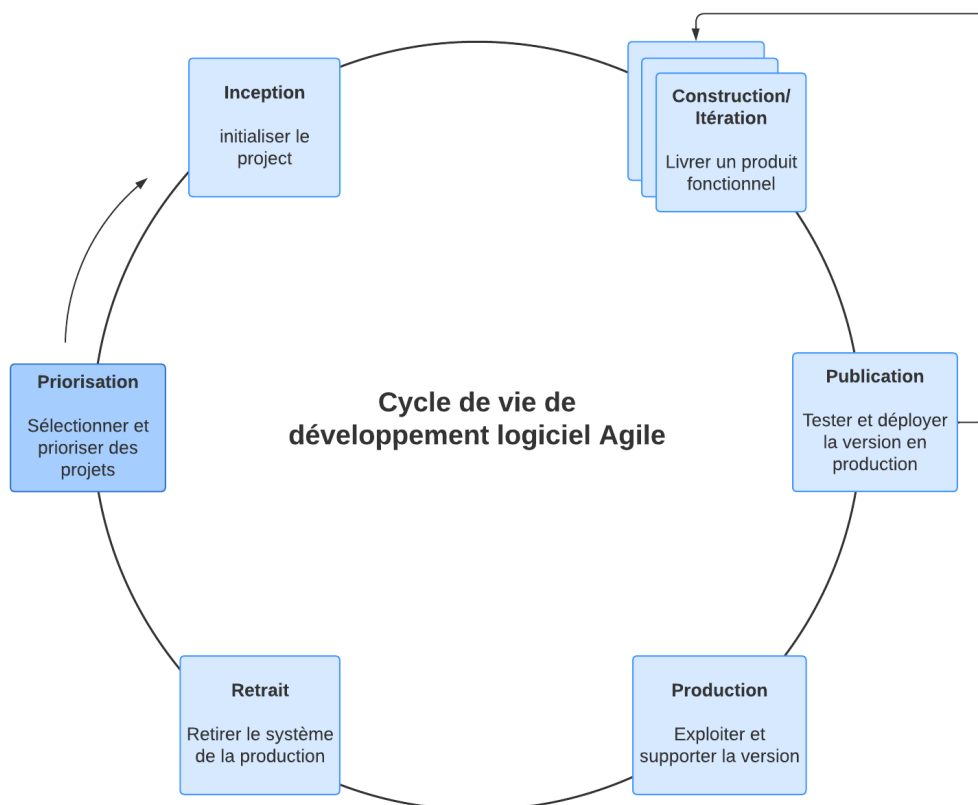


Fig. 1.2 : Cycle de vie de développement d'un logiciel agile

1.2.2.1 Méthodologie Scrum

La méthodologie Scrum est de loin la méthode Agile la plus utilisée dans le monde, offrant un cadre organisé pour superviser la création d'applications complexes. Cette approche intègre de différentes pratiques centralisées sur la flexibilité et la collaboration avec le client afin de s'adapter aux exigences changeantes tout en reposant sur la maintenance d'un « Backlog », qui décrit les changements ou améliorations à apporter sous forme des « user stories ».

1.2.2.2 Caractéristiques de Scrum

Les livraisons fréquentes sont une caractéristique clé de Scrum. Pour atteindre cet objectif, Scrum emploie des itérations de développement, connues sous le nom de « sprints », qui durent généralement de 2 à 4 semaines. Durant ces sprints, les opérations de développement traite activement les problématiques métier en cours, abordant les dossiers d'analyse et de conception afin de réaliser les phases de développement, de test et de déploiement[4].

1.2.2.3 Rôles dans Scrum

La méthodologie Scrum repose sur trois rôles essentiels :

- Product Owner : chargé de déterminer et de donner la priorité aux éléments du backlog du produit. Celle-ci informe l'équipe de développement des besoins et des exigences du client et prend les décisions concernant les fonctionnalités à fournir.
- Scrum Master : garantir que l'équipe comprend et suit les principes et les méthodes de Scrum. Celui-ci facilite la communication des membres de l'équipe et soutient l'équipe dans l'accomplissement de ses objectifs.
- Équipe de développement : sont chargés de transférer les éléments du backlog produit en fonctionnalités livrables, celles-ci collaborent pour atteindre les objectifs du sprint et sont autonomes dans leur approche de travail.

La figure 1.3 ci-dessous illustre le processus de la méthode Scrum :

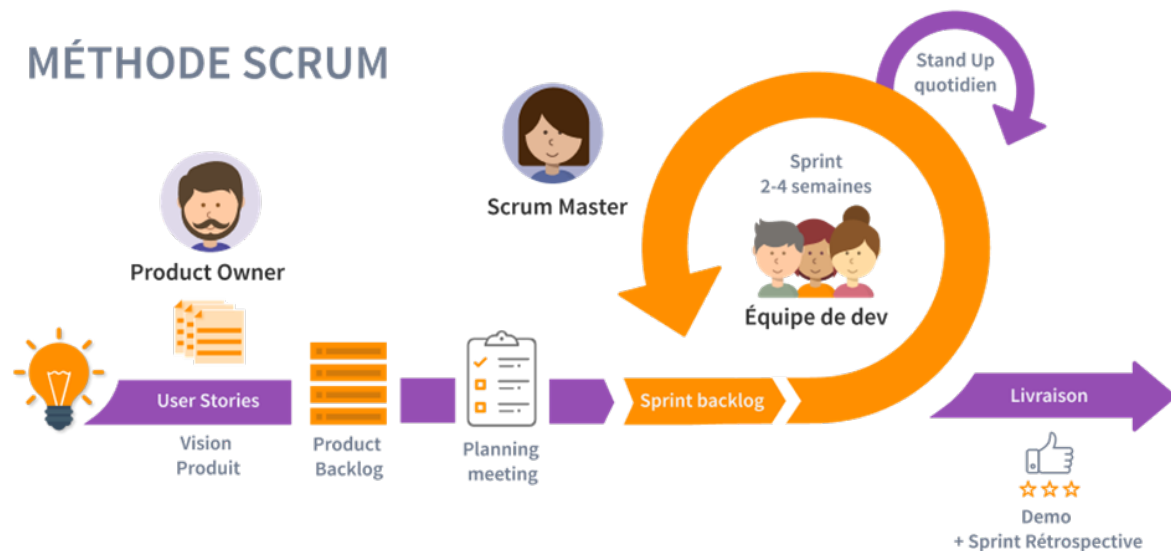


Fig. 1.3 : Processus de la méthode Scrum

1.3 Langage de Modélisation

Le choix du langage de modélisation dans le présent projet a été porté sur UML. Il est justifié par sa standardisation, sa capacité à clarifier et à documenter les spécifications du système, ainsi que par son utilité dans l'analyse, la conception, la maintenance et la validation du système logiciel.

En effet, UML offre un ensemble de conventions visuelles pour représenter les systèmes et les processus à l'aide de diagrammes et de représentations graphiques. Cette approche simplifie la communication des concepts complexes, favorisant ainsi une collaboration fluide et une compréhension approfondie à chaque étape du développement et de la conception.

1.3.1 Présentation de UML

Le langage de modélisation UML est l'un des langages de modélisation les plus couramment employés dans le domaine de l'ingénierie logicielle. UML propose une méthode efficace pour représenter visuellement la structure, le comportement et les interactions d'un système logiciel en utilisant des diagrammes standardisés tels que les diagrammes de classes, de cas d'utilisation, de séquence et bien d'autres[5].

1.3.2 Diagrammes UML

Le langage de modélisation UML est défini par 14 types de diagrammes, distingués en deux catégories : "structure" et "comportement", afin de garantir une modélisation appropriée des caractéristiques de tout système. Afin de garantir une modélisation complète et précise des fonctionnalités de la future application, nous avons choisi d'utiliser les diagrammes UML répertoriés dans le tableau 1.2 ci-dessous :

Tab. 1.2 : Types des diagrammes UML

Diagramme	Description
Diagramme de classe	Le diagramme UML le plus largement utilisé, essentiel à toute approche orientée objet, est le diagramme de classe. Ce diagramme représente des entités spécifiques du système applicatif sous forme de classes. Chaque classe est composée d'attributs, définissant les caractéristiques des objets créés à partir de ces classes, ainsi que de méthodes, définissant les actions que ces objets peuvent effectuer.
Diagramme de cas d'utilisation	Le diagramme de cas d'utilisation représente une fonctionnalité spécifique dans un système applicatif et met en évidence les liens et les interactions avec ses différentes fonctionnalités à l'aide d'acteurs internes et externes.
Diagramme de séquence	Le diagramme de séquence illustre comment les objets d'un scénario spécifique interagissent les uns avec les autres dans un ordre chronologique.

Conclusion

Dans ce premier chapitre, nous avons présenté le contexte général du projet Planora, en mettant en lumière l'étude de l'existant, la critique des solutions disponibles ainsi que la solution proposée. Nous avons également exposé le choix méthodologique adopté, basé sur une approche hybride Water-Scrum-Fall, ainsi que le langage de modélisation UML retenu pour la conception du système. Dans le chapitre suivant, nous approfondirons l'analyse et la spécification des besoins fonctionnels et non fonctionnels de l'application.

2

Analyse et Spécification des besoins

Introduction

Dans ce chapitre, nous identifierons les acteurs du projet et définirons les besoins fonctionnels et non fonctionnels. Ensuite, nous détaillerons toutes les fonctionnalités du projet à l'aide d'un diagramme de cas d'utilisation global et de ses raffinements. Enfin, nous présenterons le planning sous la forme d'un diagramme de Gantt et décrirons l'approche Scrum adoptée pour la gestion du projet.

2.1 Spécification des besoins

Dans cette section, nous présenterons les différents acteurs de l'application, ainsi que les besoins fonctionnels et non fonctionnels identifiés pour le projet.

2.1.1 Identification des acteurs

L'acteur, qu'il soit interne ou externe, physique ou logiciel, est une entité capable d'interagir directement avec le système applicatif. Dans ce cadre, nous avons identifié quatre acteurs, comme présenté dans le tableau 2.1 ci-dessous :

Acteurs	Description
Administrateur	Dispose d'un accès aux fonctionnalités d'administration de la plateforme. Il peut gérer les utilisateurs, attribuer les rôles, activer ou désactiver les comptes, créer et supprimer des espaces de travail, inviter des membres et supprimer des projets.
Chef de projet	Responsable de la gestion des projets et de la collaboration au sein des espaces de travail auxquels il appartient. Il peut consulter les espaces accessibles, créer et modifier des projets, inviter des membres, gérer les tâches, le backlog, les sprints, les réunions et suivre l'avancement du projet.
Membre	Représente un utilisateur appartenant à un espace de travail ou à un projet. Il peut consulter les projets auxquels il est affecté, participer aux discussions, consulter et mettre à jour les tâches, commenter les éléments du backlog, suivre les sprints et accéder aux réunions selon ses droits d'accès.
Utilisateur non authentifié	Peut accéder aux interfaces d'inscription et de connexion afin de créer un compte ou de s'authentifier pour accéder aux fonctionnalités de l'application.

**Tab. 2.1 : Identification des Acteurs et leurs Fonctions
dans le Système Applicatif**

Remarque : Les fonctionnalités accessibles dépendent du rôle de l'utilisateur ainsi que de son appartenance à un espace de travail ou à un projet.

2.1.2 Besoins fonctionnels

La mise en œuvre de la future application Planora doit répondre aux besoins fonctionnels suivants :

- **Gestion d'inscription** :
 - Création d'un nouveau compte utilisateur.
 - Enregistrement des informations personnelles de l'utilisateur.

- **Gestion de l'authentification :**

- Connexion sécurisée à l'application.
- Gestion de la session utilisateur.

- **Gestion des utilisateurs et de leurs droits d'accès :**

- Consultation de la liste des utilisateurs.
- Attribution des rôles aux utilisateurs.
- Activation et désactivation des comptes utilisateurs.

- **Gestion des espaces de travail :**

- Création et consultation des espaces de travail.
- Invitation des membres à rejoindre un espace de travail.
- Acceptation ou refus des invitations reçues.
- Suppression des membres d'un espace de travail.

- **Gestion des projets :**

- Création, consultation, modification et suppression des projets.
- Ajout et suppression des membres d'un projet.
- Invitation des membres à rejoindre un projet.

- **Gestion du backlog :**

- Création, modification et suppression des éléments du backlog.
- Affectation des éléments du backlog aux membres.
- Définition de la priorité, de la complexité, des story points et de la date d'échéance.
- Ajout de commentaires, de liens, de branches, de commits et de pièces jointes.

- **Gestion des tâches et sous-tâches :**

- Création, consultation, modification et suppression des tâches.
- Affectation des tâches aux membres du projet.

- Suivi de l'état d'avancement des tâches et des sous-tâches.
- **Gestion des sprints :**
 - Création, consultation, modification et suppression des sprints.
 - Association des éléments du backlog à un sprint.
 - Suivi de l'avancement des sprints à travers un tableau de bord agile.
- **Gestion de la collaboration :**
 - Échange de messages entre les membres d'un projet.
 - Consultation des conversations liées aux projets.
 - Épinglage des messages importants.
- **Gestion des réunions :**
 - Création et consultation des réunions.
 - Planification des réunions dans un calendrier.
 - Définition des membres concernés par chaque réunion.
- **Suivi et analyse de la santé de l'équipe :**
 - Analyse des messages échangés entre les membres.
 - Détection du stress, de la frustration et de l'ambiance générale.
 - Génération d'indicateurs et d'alertes liés à la santé de l'équipe.
- **Consultation du tableau de bord :**
 - Visualisation de l'avancement des projets.
 - Consultation des statistiques globales et des indicateurs de suivi.

2.1.3 Besoins non fonctionnels

Les besoins non fonctionnels sont tout aussi cruciaux que les besoins fonctionnels dans tout projet, affectant la qualité, la sécurité et l'expérience utilisateur globale. Ils sont essentiels pour garantir la performance et la satisfaction des utilisateurs finaux.

- **Ergonomie** : L'application doit disposer d'une interface ergonomique, intuitive et conforme aux bonnes pratiques UI/UX, afin d'offrir une expérience utilisateur optimale.
- **Fiabilité** : L'application doit garantir une disponibilité élevée, des temps de réponse réduits et une gestion efficace des erreurs.
- **Maintenabilité** : Le code doit être organisé de manière claire, modulaire et documentée, afin de faciliter sa compréhension, son évolution et sa maintenance par d'autres développeurs.
- **Sécurité** : L'application doit intégrer un dispositif de sécurité cohérent garantissant l'intégrité, la disponibilité et la confidentialité des données, ainsi que l'authentification des utilisateurs.

2.2 Diagramme de cas d'utilisation global

Le diagramme de cas d'utilisation global présenté dans la figure 2.1 offre une vue d'ensemble des interactions des acteurs avec les diverses fonctionnalités de l'application cible, permettant ainsi une compréhension rapide des fonctionnalités et des besoins des utilisateurs.

Fig. 2.1 : Diagramme de cas d'utilisation globale

2.3 Raffinement des cas d'utilisation

2.3.1 Gestion des utilisateurs et authentification

La figure 2.2 ci-dessous illustre le diagramme de raffinement de la gestion des utilisateurs et de l'authentification.

Fig. 2.2 : Diagramme de cas d'utilisation « Gestion des utilisateurs et authentification »

Les tableaux ci-dessous présentent une description textuelle des cas d'utilisation liés à la gestion des utilisateurs.

2.3.1.1 Utilisateur

S'authentifier	
Objectif	Permettre à l'utilisateur d'accéder à l'application Planora selon son rôle.
Acteur(s)	Administrateur, ProjectManager, Membre
Précondition	L'utilisateur doit posséder un compte valide.
Scénario	1. L'utilisateur accède à la page de connexion. 2. L'utilisateur saisit son adresse email et son mot de passe. 3. Le système vérifie les informations d'identification. 4. Si les informations sont valides, le système génère une session et redirige l'utilisateur vers le tableau de bord. 5. Si les informations sont incorrectes, le système affiche un message d'erreur.
Postcondition	L'utilisateur est authentifié et accède aux fonctionnalités autorisées selon son rôle.

Tab. 2.2 : Description textuelle du cas d'utilisation
« S'authentifier »

2.3.1.2 Administrateur

Gérer les utilisateurs	
Objectif	Administrer les comptes utilisateurs et gérer leurs rôles dans l'application.
Acteur(s)	Administrateur
Précondition	L'administrateur doit être authentifié.
Scénario	1. L'administrateur accède à la liste des utilisateurs. 2. Le système affiche les comptes enregistrés dans l'application. 3. L'administrateur sélectionne un utilisateur. 4. L'administrateur peut modifier ses informations, lui attribuer un rôle, activer ou désactiver son compte. 5. Le système enregistre les modifications effectuées.

Gérer les utilisateurs	
Postcondition	Les informations ou les droits d'accès de l'utilisateur sont mis à jour.

Tab. 2.3 : Description textuelle du cas d'utilisation
« Gérer les utilisateurs »

2.3.2 Gestion des espaces de travail

La figure 2.3 ci-dessous illustre le diagramme de raffinement de la gestion des espaces de travail.

Fig. 2.3 : Diagramme de cas d'utilisation « Gestion des espaces de travail »

Gérer un espace de travail	
Objectif	Créer, consulter et organiser les espaces de travail de l'application.
Acteur(s)	Administrateur, ProjectManager, Membre
Précondition	L'utilisateur doit être authentifié.
Scénario	1. L'utilisateur accède à la section des espaces de travail. 2. Le système affiche les espaces de travail accessibles. 3. L'administrateur peut créer un nouvel espace de travail. 4. L'administrateur ou le ProjectManager peut inviter des membres selon les droits accordés. 5. Le membre invité peut accepter ou refuser l'invitation reçue. 6. Le système met à jour la liste des membres de l'espace de travail.
Postcondition	L'espace de travail est créé ou mis à jour avec les membres concernés.

Tab. 2.4 : Description textuelle du cas d'utilisation
« Gérer un espace de travail »

2.3.3 Gestion des projets

La figure 2.4 ci-dessous illustre le diagramme de raffinement de la gestion des projets.

Fig. 2.4 : Diagramme de cas d'utilisation « Gestion des projets »

Gérer un projet	
Objectif	Permettre la création, la modification et le suivi des projets.
Acteur(s)	Administrateur, ProjectManager, Membre
Précondition	L'utilisateur doit être authentifié et appartenir à l'espace de travail concerné.
Scénario	1. L'utilisateur accède à la liste des projets. 2. Le système affiche les projets accessibles. 3. Le ProjectManager crée ou modifie un projet. 4. Le ProjectManager invite des membres à rejoindre le projet. 5. Les membres acceptent ou refusent l'invitation. 6. Le système met à jour les informations du projet et sa liste des membres.
Postcondition	Le projet est créé, modifié ou mis à jour avec les membres affectés.

**Tab. 2.5 : Description textuelle du cas d'utilisation
« Gérer un projet »**

2.3.4 Gestion du backlog, des tâches et des sprints

La figure 2.5 ci-dessous illustre le diagramme de raffinement de la gestion du backlog, des tâches et des sprints.

Fig. 2.5 : Diagramme de cas d'utilisation « Gestion du backlog, des tâches et des sprints »

Gérer le backlog et les sprints	
Objectif	Organiser les éléments du backlog, les tâches et les sprints d'un projet.
Acteur(s)	ProjectManager, Membre
Précondition	L'utilisateur doit être authentifié et avoir accès au projet concerné.
Scénario	1. L'utilisateur accède au backlog du projet. 2. Le système affiche les éléments du backlog existants. 3. L'utilisateur peut créer, modifier ou supprimer un élément du backlog. 4. L'utilisateur peut affecter un élément à un membre, définir sa priorité, sa complexité, ses story points et sa date d'échéance. 5. L'utilisateur crée un sprint et y associe des éléments du backlog. 6. Le système met à jour l'état d'avancement des tâches et des sprints.
Postcondition	Le backlog, les tâches ou les sprints sont mis à jour selon les actions effectuées.

Tab. 2.6 : Description textuelle du cas d'utilisation
« Gérer le backlog et les sprints »

2.3.5 Gestion de la collaboration et des réunions

La figure 2.6 ci-dessous illustre le diagramme de raffinement de la gestion de la collaboration et des réunions.

Fig. 2.6 : Diagramme de cas d'utilisation « Gestion de la collaboration et des réunions »

Collaborer au sein d'un projet	
Objectif	Faciliter la communication entre les membres d'un projet et organiser les réunions.

Collaborer au sein d'un projet	
Acteur(s)	ProjectManager, Membre
Précondition	L'utilisateur doit être authentifié et appartenir au projet concerné.
Scénario	1. L'utilisateur accède à l'espace de discussion du projet. 2. Le système affiche les conversations associées au projet. 3. L'utilisateur envoie ou consulte les messages échangés avec les membres. 4. L'utilisateur peut consulter les réunions planifiées dans le calendrier. 5. Le ProjectManager peut créer une réunion et définir les membres concernés. 6. Le système enregistre la réunion et l'affiche dans le calendrier.
Postcondition	Les messages ou les réunions du projet sont enregistrés et consultables par les membres concernés.

Tab. 2.7 : Description textuelle du cas d'utilisation
« Collaborer au sein d'un projet »

2.3.6 Suivi de la santé de l'équipe

La figure 2.7 ci-dessous illustre le diagramme de raffinement du suivi de la santé de l'équipe.

Fig. 2.7 : Diagramme de cas d'utilisation « Suivi de la santé de l'équipe »

Analyser la santé de l'équipe	
Objectif	Analyser les messages échangés afin d'évaluer l'ambiance générale de l'équipe.
Acteur(s)	ProjectManager, Membre
Précondition	L'utilisateur doit être authentifié et avoir accès au projet concerné.

Analyser la santé de l'équipe	
Scénario	1. L'utilisateur accède à la page de suivi de la santé de l'équipe. 2. Le système récupère les messages récents échangés dans le projet. 3. Le module d'intelligence artificielle analyse les messages. 4. Le système classe les messages selon leur sentiment : positif, neutre, stressé ou frustré. 5. Le système calcule un score global et affiche les indicateurs de santé de l'équipe. 6. En cas de stress ou de frustration élevés, le système affiche des alertes.
Postcondition	Les indicateurs et alertes liés à la santé de l'équipe sont affichés à l'utilisateur.

Tab. 2.8 : Description textuelle du cas d'utilisation
« Analyser la santé de l'équipe »

Conclusion

Dans le deuxième chapitre, nous avons nous avons effectué une analyse et une spécification des besoins en identifiant les acteurs clés du projet. Ensuite nous avons défini les exigences fonctionnelles et non fonctionnelles. Enfin, nous avons présenté une illustration du diagramme d'utilisation global ainsi que ses raffinements.

3

Conception

Introduction

Dans ce chapitre, nous explorons les différentes fonctionnalités de l'application à travers des diagrammes de classe et de séquences, décrivant ainsi la structure et les interactions entre les différents acteurs de l'application.

3.1 Outil de conception

3.1.1 StarUML

StarUML, développé par MKLab, est un outil de modélisation UML puissant et convivial. Il offre une interface intuitive pour créer des diagrammes de classe, de séquence et d'autres types de diagrammes UML. StarUML prend en charge la génération de code à partir des diagrammes, facilitant ainsi la transition entre la conception et le développement. [6] La figure 3.1 ci-dessous illustre la méthodologie hybride adoptée :



Fig. 3.1 : Logo de StarUML

3.2 Conception détaillée

3.2.1 Diagramme de classes général

Le diagramme de classes général de l'application Planora est élaboré à partir des entités du domaine définies dans le backend. Il permet de représenter la structure principale du système ainsi que les relations entre les utilisateurs, les espaces de travail, les projets, les tâches, le backlog, les sprints, la messagerie et les réunions.

Le diagramme de classes général comprend principalement :

- **Classe "ApplicationUser"** : Représente l'utilisateur de l'application. Elle hérite de la classe IdentityUser et contient les informations personnelles de l'utilisateur, son état d'activation, son jeton de rafraîchissement ainsi que ses relations avec les projets, les espaces de travail, les tâches, les commentaires, les messages et les sessions de discussion.
- **Classe "Workspace"** : Représente un espace de travail. Elle contient le nom, la description, le propriétaire, le ProjectManager, les membres, les projets associés et les invitations envoyées.
- **Classe "WorkspaceUser"** : Représente l'association entre un utilisateur et un espace de travail. Elle contient l'identifiant de l'utilisateur, l'identifiant de l'espace de travail et la date d'adhésion.
- **Classe "WorkspaceInvitation"** : Représente une invitation envoyée à un utilisateur pour rejoindre un espace de travail. Elle contient l'email de l'invité, le token, la date d'expiration, l'état d'acceptation et le rôle proposé.
- **Classe "Project"** : Représente un projet appartenant à un espace de travail. Elle contient le nom, la description, les dates de début et de fin, le ProjectManager, les membres du projet, les sprints, les éléments du backlog et les sessions de discussion.
- **Classe "ProjectUser"** : Représente l'association entre un utilisateur et un projet. Elle contient l'identifiant du projet, l'identifiant de l'utilisateur et la date d'affectation.

- **Classe "ProjectInvitation"** : Représente une invitation envoyée à un utilisateur pour rejoindre un projet. Elle contient l'utilisateur invité, l'utilisateur émetteur, la date d'expiration, la date de réponse et l'état d'acceptation.
- **Classe "BacklogItem"** : Représente un élément du backlog. Elle contient le titre, la description, la priorité, le statut, la complexité, les story points, la date d'échéance, le sprint associé et l'utilisateur assigné.
- **Classe "SubTask"** : Représente une sous-tâche liée à un élément du backlog. Elle contient le titre, l'état d'achèvement et l'identifiant de l'élément du backlog associé.
- **Classe "Sprint"** : Représente une itération de travail. Elle contient le nom, l'objectif, les dates de début et de fin, le statut, le projet associé et les tâches incluses dans le sprint.
- **Classe "TaskItem"** : Représente une tâche du projet. Elle contient le titre, la description, le statut, la priorité, le pourcentage d'avancement, la date d'échéance, le projet, le sprint et l'utilisateur assigné.
- **Classe "Comment"** : Représente un commentaire rédigé par un utilisateur. Il peut être associé soit à une tâche, soit à un élément du backlog.
- **Classe "ChatSession"** : Représente une session de discussion liée à un projet. Elle contient le titre, le créateur de la session et la collection des messages associés.
- **Classe "ChatMessage"** : Représente un message envoyé dans une session de discussion. Elle contient le contenu, l'expéditeur, le type du message, les informations de modification, les pièces jointes et les réactions.
- **Classe "ChatMessageReaction"** : Représente une réaction à un message. Elle contient l'utilisateur, le message concerné et l'emoji utilisé.
- **Classe "MeetingEvent"** : Représente une réunion planifiée. Elle contient le projet concerné, le titre, la date de planification, le créateur, l'option de réunion en ligne et les membres visibles.
- **Classe "PinnedMessage"** : Représente un message épinglé dans un projet. Elle contient le contenu du message, une note éventuelle, l'utilisateur qui l'a épinglé et la date d'épinglage.

- **Classe "DailyCheckIn"** : Représente le suivi quotidien d'un membre. Elle contient le niveau d'énergie, les heures disponibles, l'existence d'un blocage, une note éventuelle, l'utilisateur et le projet concernés.
- **Classes "BacklogAttachment", "BacklogBranch", "BacklogCommit", "BacklogLink" et "BacklogWebLink"** : Représentent les éléments complémentaires liés au backlog, tels que les pièces jointes, les branches, les commits, les liens entre éléments du backlog et les liens web externes.

La figure 3.2 illustre le diagramme de classes général de l'application Planora.

Fig. 3.2 : Diagramme de classes général de l'application Planora

3.2.2 Gestion de l'authentification et des utilisateurs

3.2.2.1 Diagramme de séquence

Au niveau de la page de connexion, l'utilisateur saisit son adresse email et son mot de passe afin d'accéder à l'application Planora. Le système vérifie les informations d'identification, puis génère un jeton d'authentification si les données saisies sont correctes.

Après authentification, l'utilisateur est redirigé vers le tableau de bord et accède aux fonctionnalités autorisées selon son rôle : Administrateur, ProjectManager ou Member.

L'administrateur peut également gérer les comptes utilisateurs, modifier leurs informations, attribuer des rôles, activer ou désactiver un compte.

La figure 3.3 illustre le diagramme de séquence de la gestion de l'authentification et des utilisateurs.

Fig. 3.3 : Diagramme de séquence de la gestion de l'authentification et des utilisateurs

3.2.3 Gestion des espaces de travail et des projets

3.2.3.1 Diagramme de séquence

L'administrateur crée un espace de travail en renseignant les informations nécessaires, telles que le nom et la description. Une fois l'espace créé, il peut inviter des membres à le rejoindre. Les utilisateurs invités reçoivent une invitation qu'ils peuvent accepter ou refuser.

Après l'organisation de l'espace de travail, le ProjectManager peut créer un projet, définir ses informations principales, affecter des membres et suivre son avancement. Les membres ajoutés au projet peuvent ensuite accéder aux fonctionnalités qui leur sont autorisées.

La figure 3.4 illustre le diagramme de séquence de la gestion des espaces de travail et des projets.

Fig. 3.4 : Diagramme de séquence de la gestion des espaces de travail et des projets

3.2.4 Gestion du backlog, des tâches et des sprints

3.2.4.1 Diagramme de séquence

Le ProjectManager ou un membre autorisé accède au backlog d'un projet afin de créer de nouveaux éléments. Chaque élément du backlog peut contenir un titre, une description, une priorité, une complexité, des story points, une date d'échéance et un membre assigné. Les éléments du backlog peuvent être associés à un sprint afin d'organiser le travail selon une approche agile. Les membres peuvent ensuite suivre l'état d'avancement des tâches, ajouter des commentaires et mettre à jour les informations nécessaires.

La figure 3.5 illustre le diagramme de séquence de la gestion du backlog, des tâches et des sprints.

Fig. 3.5 : Diagramme de séquence de la gestion du backlog, des tâches et des sprints

3.2.5 Gestion de la collaboration et des réunions

3.2.5.1 Diagramme de séquence

Les membres d'un projet peuvent communiquer à travers un espace de messagerie intégré. Lorsqu'un utilisateur envoie un message, celui-ci est enregistré et rattaché à une session de discussion liée au projet. Les autres membres concernés peuvent consulter les messages et suivre les échanges.

La plateforme permet également de planifier des réunions à travers un calendrier. L'utilisateur renseigne les informations de la réunion, telles que le titre, la date, l'heure et les membres concernés. Le système enregistre ensuite l'événement et l'affiche dans le calendrier du projet.

La figure 3.6 illustre le diagramme de séquence de la gestion de la collaboration et des réunions.

Fig. 3.6 : Diagramme de séquence de la gestion de la collaboration et des réunions

3.2.6 Suivi de la santé de l'équipe par intelligence artificielle

3.2.6.1 Diagramme de séquence

Le suivi de la santé de l'équipe repose sur un module d'intelligence artificielle permettant d'analyser les messages échangés entre les membres d'un projet. Lorsqu'un utilisateur accède à la page dédiée, le système récupère les messages récents et les transmet au service d'analyse.

Le modèle d'intelligence artificielle classe les messages selon plusieurs sentiments, notamment positif, neutre, stressé ou frustré. À partir de ces résultats, le système calcule un score global, génère une distribution des sentiments et affiche des alertes lorsque des signes de stress ou de frustration sont détectés.

La figure 3.7 illustre le diagramme de séquence du suivi de la santé de l'équipe par intelligence artificielle.

Fig. 3.7 : Diagramme de séquence du suivi de la santé de l'équipe par intelligence artificielle

Conclusion

Dans ce chapitre, nous avons présenté la conception détaillée de l'application Planora à travers les principaux diagrammes de séquence décrivant les interactions entre les acteurs et le système. Nous avons également présenté le diagramme de classes général afin de modéliser la structure globale de l'application et les relations entre ses principales entités.

4

Réalisation

Introduction

Dans ce chapitre, nous présenterons les outils de développement utilisés, ainsi que l'architecture technique adoptée pour le processus de développement. Ensuite, nous détaillerons les incréments réalisés à chaque sprint, illustrant les différentes fonctionnalités du projet.

4.1 Environnement de développement

4.1.1 SQL Server Management Studio

SQL Server Management Studio (SQL Server Management Studio (SSMS)), développé par Microsoft en 2005, est un environnement de gestion intégré pour les bases de données SQL Server. Il offre une interface graphique complète pour la gestion, le développement et l'administration des bases de données SQL Server, facilitant ainsi les tâches telles que la création de requêtes, la gestion des objets de base de données et la surveillance des performances. [7]



Fig. 4.1 : Logo de SQL Server Management Studio

4.1.2 Visual Studio

Visual Studio (Visual Studio (VS)), développé par Microsoft en 2005, est un environnement de développement intégré (IDE) puissant. Il propose une suite complète de fonctionnalités concentrées pour faciliter la création, la modification et le débogage des applications .NET. [8]



Fig. 4.2 : Logo de Visual Studio

4.1.3 Visual Studio Code

Visual Studio Code (Visual Studio Code (VSCode)), développé par Microsoft en 2015, est un éditeur de code qui peut être étendu avec des extensions pour le transformer en un environnement de développement intégré (IDE) complet. [9]



Fig. 4.3 : Logo de Visual Studio Code

4.1.4 Postman

Postman, développé par Postdot Technologies en 2012, est une plateforme de développement et de test d'API. Cet outil offre une interface intuitive et un ensemble de fonctionnalités pour simplifier la création, le test et le débogage des API. [10]



Fig. 4.4 : Logo de Postman

4.2 Environnement logiciel

4.2.1 Microsoft SQL Server

SQL Server, initialement lancé en 1989, est un système de gestion de bases de données relationnelles développé par Microsoft. Il offre une plateforme complète pour la gestion et l'analyse des données, avec des fonctionnalités avancées de sécurité, de performance et de disponibilité. [11]



Fig. 4.5 : Logo de Microsoft SQL Server

4.2.2 Dotnet

Dotnet (DOTNET), développé par Microsoft et lancé en 2002, est une plateforme de développement logiciel qui permet de créer et d'exécuter des applications sur plusieurs systèmes d'exploitation. Elle offre une grande variété de langages de programmation et d'outils pour faciliter le développement d'applications modernes.[12]



Fig. 4.6 : Logo de Dotnet

4.2.3 LaTeX

LaTeX, développé par Leslie Lamport en 1985, est un système de composition de documents qui permet de créer des documents de haute qualité typographique. Il est largement utilisé dans les domaines académiques et scientifiques pour la rédaction d'articles, de rapports et de thèses.[13]

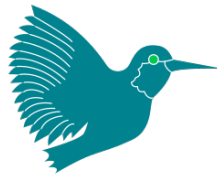


Fig. 4.7 : Logo de LaTeX

4.3 Pilotage du cycle de vie de développement avec Scrum

Dans cette section, nous mettrons en lumière l'application de la méthode Scrum pour piloter le projet. Nous présenterons les membres de l'équipe ainsi que leurs rôles respectifs, puis nous examinerons le backlog du produit.

4.3.1 Équipe Scrum et rôles

La création d'une équipe Scrum solide est une étape essentielle pour la réussite d'un projet.

Les membres de l'équipe Scrum ainsi que leurs rôles sont illustrés dans la figure 4.8 ci-dessous :

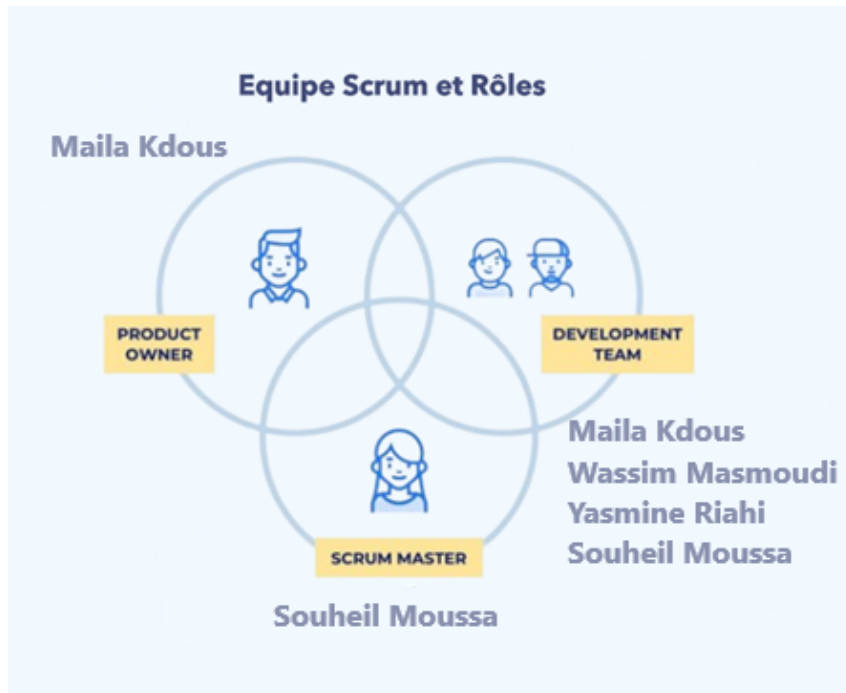


Fig. 4.8 : Équipe Scrum et rôles

4.4 Architecture technique

4.4.1 Architecture générale

L'application web conçue sera déployée selon une architecture à 3 niveaux (3-Tiers). Cette architecture est représentée par la figure 4.9 :



Fig. 4.9 : Architecture générale

4.4.2 Architecture de l'environnement

Dans cette section, nous explorerons l'architecture technique de l'application, en décrivant ses différentes composantes techniques, à savoir, le Back End et le Front End.

4.4.2.1 Architecture du Back End

Le Back End de l'application Planora est développé avec ASP.NET Core et repose sur une architecture basé sur la "Clean Architecture". Cette organisation permet de séparer les responsabilités entre la présentation des services API, la logique applicative, la logique métier et l'accès aux données. Elle facilite également la maintenance, l'évolution et le test de l'application.

L'architecture Back End est composée principalement des couches suivantes :

- **Couche Application Programming Interface (API) "Planora"** : Cette couche représente le point d'entrée de l'application côté serveur. Elle contient les contrôleurs API REST qui reçoivent les requêtes HTTP provenant du Front End, valident l'accès de l'utilisateur, appellent les services nécessaires et retournent les réponses sous forme d'objets JavaScript Object Notation (JSON). Elle contient également la configuration générale de l'application, le middleware de gestion des erreurs et le hub SignalR (SignalR) utilisé pour la communication en temps réel.
- **Couche Application "Planora.Application"** : Cette couche regroupe les contrats et les objets de transfert de données utilisés par l'application. Elle contient les interfaces des services, les DTOs, les validateurs et les profils de mapping. Son rôle est de définir les opérations disponibles sans dépendre directement de leur implémentation technique.
- **Couche Domaine "Planora.Domain"** : Cette couche contient les entités principales du système, telles que ApplicationUser, Workspace, Project, BacklogItem, Sprint, TaskItem, Comment et ChatMessage. Elle représente le cœur métier de l'application et définit la structure des données manipulées par Planora.
- **Couche Infrastructure "Planora.Infrastructure"** : Cette couche contient les implémentations concrètes des services, les repositories, le contexte de base de données, les configurations Entity Framework Core, les migrations et les services techniques comme l'authentification par JSON Web Token (JWT), l'envoi d'emails et

l'intégration avec la base de données. Elle assure la communication avec SQL Server et réalise les traitements nécessaires à la persistance des données.

- **Module IA "planora-ai"** : Ce module est développé en Python avec Flask. Il expose une API dédiée à l'analyse des messages échangés entre les membres d'un projet. Le modèle d'intelligence artificielle classe les messages selon leur sentiment et renvoie les résultats au Back End ou au Front End afin d'afficher les indicateurs de santé de l'équipe.

Ainsi, cette architecture permet de garantir une séparation claire entre les différentes responsabilités du système.

4.4.2.2 Architecture du Front End

L'architecture utilisée dans la partie Front End est une architecture basée sur les composants, comprenant les différentes parties suivantes :

- **Composants** : Ce sont les blocs de construction de l'interface utilisateur (UI). Chaque composant représente une partie de l'UI et est responsable de sa propre logique afin de réaliser une fonction spécifique.
- **Services** : Ils sont utilisés pour encapsuler la logique métier qui n'appartient pas spécifiquement à un composant particulier, telle que gérer les appels API REST, et les fonctionnalités transversales à plusieurs composants.
- **Modules** : Ce sont des regroupements logiques de composants, de services et d'autres fonctionnalités de l'application afin de permettre une organisation du code de manière modulaire.

Il convient de noter que cette architecture favorise la création d'applications modulaires grâce à des composants interchangeables et réutilisables.

4.4.3 Tests et validation

Afin de tester efficacement les fonctionnalités de l'application et assurer son bon fonctionnement, nous avons adopté l'utilisation de Postman.

La figure 4.10 illustre le processus d'envoi d'une requête depuis Postman vers l'application pour tester ses points d'accès.

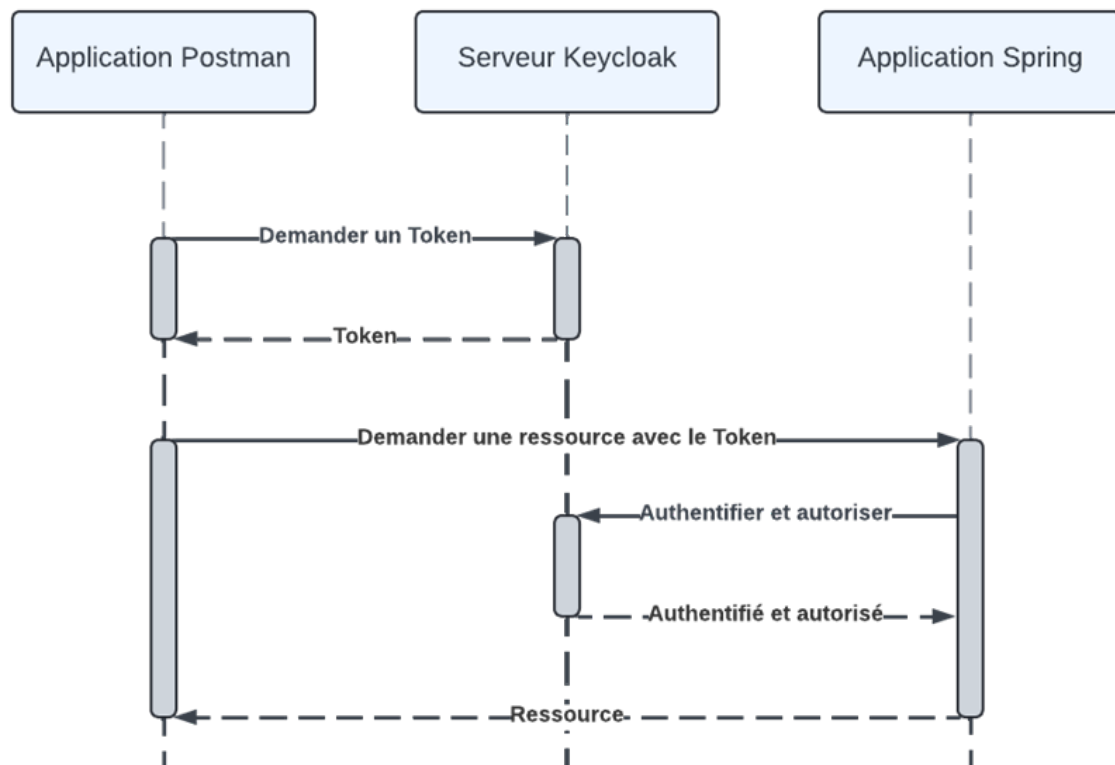


Fig. 4.10 : Processus de test des points d'accès sécurisés par Keycloak

4.4.4 Backlog du produit

Dans cette section, nous présenterons les différentes fonctionnalités du produit et décrirons les scénarios d'utilisation associés à travers des stories utilisateur. Cette approche permettra de formaliser efficacement les besoins des utilisateurs et de prioriser le travail de développement.

Ci-dessous, nous détaillons la signification des différents termes utilisés dans le backlog produit :

- **ID** : Identifiant unique de chaque fonctionnalité.
- **Fonctionnalité** : Une caractéristique particulière proposée par l'application.
- **User Story** : Une fonctionnalité que l'utilisateur final souhaite accomplir avec le système applicatif.
- **Priorité** : Niveau d'importance ou d'urgence attribué à une User Story.

Le tableau 4.2 présente le backlog du produit ainsi que la priorisation des fonctionnalités.

4.4.4.1 Légende des priorités

Tab. 4.1 : Légende des priorités du backlog

Priorité	Signification
Critique	À réaliser en premier car l'élément conditionne plusieurs autres tâches.
Élevé	À traiter rapidement car l'impact fonctionnel ou technique est important.
Moyen	À planifier après les fondations principales, avec une dépendance plus limitée.
Faible	À intégrer en dernier, une fois le socle applicatif stabilisé.

4.4.4.2 Backlog du produit

Tab. 4.2 : Backlog du produit et priorisation des tâches

ID	Fonctionnalité	User Story	Priorité	Remarques
1	Authentification et inscription	En tant qu'utilisateur, je veux créer un compte et m'authentifier afin d'accéder à l'application Planora de manière sécurisée.	Critique	Fonction de base nécessaire pour accéder aux autres modules.
2	Gestion des rôles et des utilisateurs	En tant qu'administrateur, je veux gérer les comptes utilisateurs, leurs rôles et leur état d'activation afin de contrôler les accès à la plateforme.	Critique	Conditionne la sécurité et la séparation des droits entre Admin, ProjectManager et Member.
3	Gestion des espaces de travail	En tant qu'administrateur, je veux créer des espaces de travail et inviter des membres afin d'organiser les équipes dans des environnements séparés.	Critique	Structure principale de l'organisation des projets.

ID	Fonctionnalité	User Story	Priorité	Remarques
4	Gestion des projets	En tant que ProjectManager, je veux créer et gérer des projets afin de planifier le travail et suivre l'avancement de l'équipe.	Critique	Dépend de la gestion des espaces de travail et des membres.
5	Gestion du backlog	En tant que ProjectManager, je veux créer, modifier, prioriser et assigner les éléments du backlog afin d'organiser les besoins du projet.	Élevé	Module central pour la gestion agile du projet.
6	Gestion des tâches et sous-tâches	En tant que membre de projet, je veux consulter, mettre à jour et suivre mes tâches afin de contribuer efficacement à l'avancement du projet.	Élevé	Complète la gestion du backlog et permet le suivi opérationnel.
7	Gestion des sprints	En tant que ProjectManager, je veux créer des sprints et y associer des éléments du backlog afin de planifier le travail par itérations.	Élevé	Nécessaire pour organiser le projet selon une approche agile.
8	Tableau de bord	En tant qu'utilisateur autorisé, je veux consulter des statistiques et indicateurs afin de suivre l'avancement des projets et des espaces de travail.	Élevé	Permet une vision synthétique de l'état des projets.
9	Messagerie de projet	En tant que membre de projet, je veux échanger des messages avec les autres membres afin de faciliter la collaboration.	Moyen	Supporte la communication interne et alimente le module d'analyse de santé.

ID	Fonctionnalité	User Story	Priorité	Remarques
10	Gestion des réunions et calendrier	En tant que membre de projet, je veux planifier et consulter les réunions afin de mieux organiser les échanges de l'équipe.	Moyen	Complète les fonctionnalités collaboratives du projet.
11	Commentaires et pièces jointes du backlog	En tant que membre de projet, je veux ajouter des commentaires, liens et pièces jointes aux éléments du backlog afin de centraliser les informations utiles.	Moyen	Enrichit le suivi détaillé des éléments du backlog.
12	Suivi de la santé de l'équipe par IA	En tant que ProjectManager, je veux analyser les messages de l'équipe afin de détecter le stress, la frustration et l'ambiance générale du projet.	Moyen	Dépend de la messagerie et du module IA Python Flask.
13	Notifications par email	En tant qu'utilisateur, je veux recevoir des notifications importantes afin d'être informé des invitations et des affectations de tâches.	Faible	Améliore l'expérience utilisateur sans bloquer les modules principaux.
14	Check-in quotidien	En tant que membre de projet, je veux renseigner mon niveau d'énergie, mes disponibilités et mes blocages afin d'aider au suivi de l'équipe.	Faible	Fonction complémentaire au suivi de la santé de l'équipe.

4.5 Sprints de réalisation

Dans cette section, nous présentons les incréments réalisés sprint par sprint, en détaillant le backlog associé à chaque itération ainsi que les interfaces livrées. Le découpage respecte les dépendances fonctionnelles du projet : d'abord l'authentification et la gestion des utilisateurs, ensuite les espaces de travail et les projets, puis la gestion agile, la

collaboration, et enfin le module d'intelligence artificielle dédié au suivi de la santé de l'équipe.

4.5.1 Sprint 1 - Authentification et gestion des utilisateurs

4.5.1.1 Backlog du sprint

Le tableau 4.3 illustre le backlog du Sprint 1.

Tab. 4.3 : Backlog du Sprint 1 : Authentification et gestion des utilisateurs

User Story	ID	Tâches
En tant qu'utilisateur, je veux créer un compte afin d'accéder à l'application Planora.	1.1	• Créer l'interface d'inscription.
	1.2	• Implémenter l'API d'inscription côté Back End.
	1.3	• Valider les informations saisies par l'utilisateur.
En tant qu'utilisateur, je veux m'authentifier afin d'accéder aux fonctionnalités selon mon rôle.	1.4	• Créer l'interface de connexion.
	1.5	• Mettre en place l'authentification par JWT.
	1.6	• Protéger les routes du Front End avec un guard d'authentification.
En tant qu'administrateur, je veux gérer les utilisateurs et leurs rôles afin de contrôler les accès à la plateforme.	1.7	• Créer les rôles Admin, ProjectManager et Member.
	1.8	• Développer la gestion des utilisateurs côté Back End.

User Story	ID	Tâches
	1.9	• Permettre l'activation, la désactivation et l'attribution des rôles.

4.5.1.2 Interfaces livrables

La figure 4.11 illustre l'interface de connexion permettant à l'utilisateur de s'authentifier avec son adresse email et son mot de passe.

Fig. 4.11 : Sprint 1 - Interface de connexion

La figure 4.12 présente l'interface d'inscription permettant à un nouvel utilisateur de créer son compte.

Fig. 4.12 : Sprint 1 - Interface d'inscription

La figure 4.13 illustre l'interface de gestion des utilisateurs réservée à l'administrateur.

Fig. 4.13 : Sprint 1 - Interface de gestion des utilisateurs

4.5.2 Sprint 2 - Gestion des espaces de travail et des projets

4.5.2.1 Backlog du sprint

Le tableau 4.4 illustre le backlog du Sprint 2.

Tab. 4.4 : Backlog du Sprint 2 : Gestion des espaces de travail et des projets

User Story	ID	Tâches
En tant qu'administrateur, je veux créer des espaces de travail afin d'organiser les équipes et les projets.	2.1	• Créer l'entité Workspace et ses relations.
	2.2	• Développer les API de création, consultation et suppression des espaces de travail.

User Story	ID	Tâches
	2.3	• Créer les interfaces de gestion des espaces de travail.
En tant qu'administrateur ou ProjectManager, je veux inviter des membres afin de constituer une équipe de travail.	2.4	• Mettre en place le système d'invitations aux espaces de travail.
	2.5	• Permettre l'acceptation et le refus des invitations.
	2.6	• Afficher la liste des membres d'un espace de travail.
En tant que ProjectManager, je veux créer et gérer des projets afin de suivre leur avancement.	2.7	• Créer l'entité Project et ses relations avec Workspace et ApplicationUser.
	2.8	• Développer les API de création, modification et consultation des projets.
	2.9	• Créer les interfaces de liste et de détail des projets.

4.5.2.2 Interfaces livrables

La figure 4.14 illustre l'interface de consultation des espaces de travail accessibles à l'utilisateur.

Fig. 4.14 : Sprint 2 - Interface des espaces de travail

La figure 4.15 présente l'interface de gestion des projets.

Fig. 4.15 : Sprint 2 - Interface de gestion des projets

4.5.3 Sprint 3 - Gestion agile : backlog, tâches et sprints

4.5.3.1 Backlog du sprint

Le tableau 4.5 illustre le backlog du Sprint 3.

Tab. 4.5 : Backlog du Sprint 3 : Gestion agile

User Story	ID	Tâches
En tant que ProjectManager, je veux gérer le backlog afin d'organiser les besoins du projet.	3.1	• Créer l'entité BacklogItem et ses relations.
	3.2	• Développer les opérations de création, modification et suppression des éléments du backlog.
	3.3	• Ajouter la priorité, la complexité, les story points et la date d'échéance.
	3.4	• Permettre l'affectation des éléments du backlog aux membres.
En tant que membre, je veux suivre mes tâches afin de connaître mon travail à réaliser.	3.5	• Créer la gestion des tâches et des sous-tâches.
	3.6	• Permettre la mise à jour de l'état et du pourcentage d'avancement.
	3.7	• Ajouter les commentaires liés aux tâches et au backlog.
En tant que ProjectManager, je veux gérer les sprints afin de planifier le travail par itérations.	3.8	• Créer l'entité Sprint et ses relations avec Project et TaskItem.
	3.9	• Développer la création et la consultation des sprints.
	3.10	• Créer le tableau de sprint pour suivre l'avancement.

4.5.3.2 Interfaces livrables

La figure 4.16 illustre l'interface de gestion du backlog.

Fig. 4.16 : Sprint 3 - Interface de gestion du backlog

La figure 4.17 présente le tableau agile permettant de suivre l'avancement des tâches du sprint.

Fig. 4.17 : Sprint 3 - Tableau de sprint

4.5.4 Sprint 4 - Collaboration, réunions et tableau de bord

4.5.4.1 Backlog du sprint

Le tableau 4.6 illustre le backlog du Sprint 4.

Tab. 4.6 : Backlog du Sprint 4 : Collaboration et suivi

User Story	ID	Tâches
En tant que membre de projet, je veux échanger avec mon équipe afin de faciliter la collaboration.	4.1	• Créer les entités ChatSession et Chat-Message.
	4.2	• Développer l'espace de messagerie par projet.
	4.3	• Ajouter les réactions, réponses et pièces jointes aux messages.
En tant que membre de projet, je veux planifier des réunions afin d'organiser les échanges de l'équipe.	4.4	• Créer la gestion des événements de réunion.
	4.5	• Afficher les réunions dans un calendrier.
	4.6	• Permettre la sélection des membres concernés par une réunion.
En tant qu'utilisateur, je veux consulter un tableau de bord afin de suivre l'avancement global.	4.7	• Calculer les indicateurs d'avancement des projets et workspaces.
	4.8	• Créer l'interface du tableau de bord.

4.5.4.2 Interfaces livrables

La figure 4.18 illustre l'interface de messagerie du projet.

Fig. 4.18 : Sprint 4 - Interface de messagerie

La figure 4.19 présente le calendrier des réunions du projet.

Fig. 4.19 : Sprint 4 - Calendrier des réunions

4.5.5 Sprint 5 - Suivi de la santé de l'équipe par IA

4.5.5.1 Backlog du sprint

Le tableau 4.7 illustre le backlog du Sprint 5.

Tab. 4.7 : Backlog du Sprint 5 : Suivi de la santé de l'équipe par IA

User Story	ID	Tâches
En tant que ProjectManager, je veux analyser les messages de l'équipe afin de détecter l'ambiance générale du projet.	5.1	• Préparer le modèle d'analyse des sentiments.
	5.2	• Développer l'API Flask d'analyse des messages.
	5.3	• Connecter le module IA à la base de données et aux messages du projet.
En tant que membre ou Project-Manager, je veux consulter les indicateurs de santé de l'équipe afin d'identifier les situations à risque.	5.4	• Classifier les messages en positif, neutre, stressé ou frustré.
	5.5	• Calculer un score global de santé de l'équipe.
	5.6	• Afficher la distribution des sentiments, les alertes et le résumé par membre.

4.5.5.2 Interfaces livrables

La figure 4.20 illustre l'interface de Dashboard

Fig. 4.20 : Sprint 5 - Interface de tableau de bord

La figure 4.21 illustre l'interface de suivi de la santé de l'équipe.

Fig. 4.21 : Sprint 5 - Interface de suivi de la santé de l'équipe

La figure 4.22 présente le résultat de test de l'API Flask dédiée à l'analyse des sentiments.

Fig. 4.22 : Sprint 5 - Test du service IA d'analyse des sentiments

Conclusion

Dans ce chapitre, nous avons présenté l'environnement logiciel, suivi de la mise en évidence de l'architecture technique des différentes couches de l'application. Enfin, nous avons détaillé les interfaces réalisées, accompagnés des captures d'écran et des test des services implémenté.

Bibliographie

- [1] Olga Rodier. Gestion de projet hybride (waterfall et agile). <https://blog-gestion-de-projet.com/modele-hybride/>, 2024.
- [2] Sarah Laoyan. Gestion de projet avec la méthode waterfall : guide complet. <https://asana.com/fr/resources/waterfall-project-management-methodology>, septembre 2025.
- [3] Comparatif méthode agile vs waterfall, kanban ou scrum. <https://www.lucidchart.com/blog/fr/methode-agile-vs-waterfall-kanban-ou-scrum>, 2024.
- [4] Team LeHibou. Méthode agile : Le guide complet. <https://www.lehibou.com/communaute/methode-agile>, mai 2021.
- [5] Qu'est-ce que le langage uml? <https://www.lucidchart.com/pages/fr/langage-uml>, 2024.
- [6] MKLab. Staruml. <https://staruml.io/>, 2026.
- [7] Microsoft. Sql server management studio. <https://learn.microsoft.com/en-us/ssms/sql-server-management-studio-ssms>, 2026.
- [8] Microsoft. Visual studio. <https://visualstudio.microsoft.com/>, 2026.
- [9] Visual studio code. <https://code.visualstudio.com/>, 2026.
- [10] Postman. <https://www.postman.com/>, 2026.
- [11] Microsoft. Microsoft sql server. <https://www.microsoft.com/en-us/sql-server/>, 2026.
- [12] Microsoft. Dotnet. <https://dotnet.microsoft.com/>, 2026.
- [13] Latex, un langage pour éditer du texte scientifique : Overleaf. <https://paris-sorbonne.libguides.com/c.php?g=497641&p=4637541>, 2024.

ملخص

يوضع الملخص باللغة العربية هنا. يجب أن يقدّم هذا القسم وصفاً موجزاً لموضوع المشروع، الأهداف الرئيسية، والمنهجية المتبعة والنتائج المتحصل عليها. يُفضّل أن لا يتجاوز الملخص عشر أسطر.

كلمات مفتاحية: كلمة1، كلمة2، كلمة3، كلمة4، كلمة5

Résumé

Mettez le résumé en français ici. Cette section doit présenter brièvement le contexte du projet, les objectifs principaux, la méthodologie utilisée ainsi que les résultats obtenus. Merci de ne pas dépasser une dizaine de lignes.

Mots clés : mot1, mot2, mot3, mot4, mot5

Abstract

Put the English abstract here. This section should briefly describe the project context, the main objectives, the methodology used, and the obtained results. Please do not exceed ten lines.

Keywords : keyword1, keyword2, keyword3, keyword4, keyword5